

BASES DE DATOS 2 · INGENIERÍA DE SISTEMAS

Optimización de Consultas e Índices

Cómo un SGBD elige, entre miles de formas posibles, la más rápida de responder tu consulta.

Procesamiento y optimización de consultas · Estadísticas y costos · Álgebra relacional · Planes de ejecución · Índices B-tree y Hash

¿Qué vamos a poder hacer al final de esta clase?

1. **Entender el recorrido de una consulta**
   Cómo el SGBD transforma un SELECT en una estrategia de ejecución concreta.

2. **Reconocer qué usa el optimizador para decidir**
   Qué información estadística de la base le permite estimar el costo de cada alternativa.

3. **Aplicar estrategias reales**
   Leer un plan de ejecución de PostgreSQL y usar índices para acelerar una consulta.

La idea central, sin tecnicismos todavía

Optimizar una consulta es como hacer las compras de forma inteligente: filtrás por lo que te interesa antes de revisar todo, y empezás por el filtro que más achica la búsqueda.

SIN OPTIMIZAR

Cruzar los 10.000 libros con los 50.000 préstamos completos, y recién al final quedarse con los de la Editorial Sur.

≈ 500.000.000

comparaciones posibles

OPTIMIZADO

Quedarse primero con los 5 libros de la Editorial Sur, y recién ahí cruzar esos 5 con los préstamos.

≈ 250.000

comparaciones (5 × 50.000)

Mismo resultado exacto en los dos casos. Muchísima menos trabajo en el segundo — eso, en esencia, es optimizar.

De tu SQL a una respuesta: 4 pasos

1

Entender

El SGBD lee tu SQL y arma una primera representación interna (un árbol de consulta).

2

Optimizar

Prueba formas equivalentes de ese árbol y elige la que estima más barata.

3

Generar

Traduce la estrategia elegida a código de bajo nivel, ejecutable.

4

Ejecutar

Corre ese código sobre los datos reales y te devuelve el resultado.

Elmasri, R.; Navathe, S.B.: Fundamentos de Sistemas de Bases de Datos. 5ª Ed. Addison-Wesley (Cap. 15).

Dos objetivos que conviene no confundir

Procesar la consulta

Transformar el SQL en UNA estrategia de ejecución válida (que dé el resultado correcto).

Optimizar la consulta

Entre TODAS las estrategias válidas posibles, elegir la que gasta menos recursos.

Dato curioso: analizar todas las estrategias posibles para encontrar la óptima sería carísimo. El optimizador no busca la perfecta — busca una muy buena, y lo logra casi siempre.

¿Por qué confiar en el optimizador automático?

✓ **Tiene mejor información**
Conoce estadísticas reales de la base: cantidad de filas, valores distintos por columna, índices disponibles.

✓ **Se adapta solo**
Si cambian los datos o el esquema, puede re-optimizar la consulta sin que nadie se lo pida.

✓ **Ve más alternativas**
Compara muchísimas más estrategias posibles que las que un programador probaría a mano.

Aun así: escribir consultas prolijas (evitar cruces innecesarios, filtrar con criterio) le facilita muchísimo el trabajo al optimizador.

El optimizador combina dos técnicas

H Optimización heurística

Reordena las operaciones de la consulta usando reglas que, casi siempre, mejoran el rendimiento — sin necesidad de calcular costos exactos.

$ Estimación de costos

Calcula un costo aproximado (uso de disco, memoria, tiempo) para varias estrategias y elige la de menor costo estimado.

Primero se aplica la heurística para reducir las alternativas; después se comparan costos entre las que quedan.

¿Qué significa “estimar” un costo?

1

No mide: estima

Antes de ejecutar nada, calcula un número que representa cuánto trabajo — lecturas de disco, uso de CPU — llevaría un plan. No es el tiempo real, es una proyección.

2

Compara planes candidatos

Para la misma consulta arma varias alternativas — distinto orden de joins, distintos algoritmos — y estima el costo de cada una.

3

Se queda con la más barata

Elige el plan de menor costo estimado: no el “perfecto”, sino el mejor según lo que logra calcular con los datos que tiene.

¿Y de dónde saca esos números? De estadísticas que el SGBD guarda sobre cada tabla — en nuestro ejercicio, por ejemplo: EMPLEADO (5.000 filas), PROYECTO (200 filas), TRABAJA_EN (30.000 filas). Veamos qué guarda exactamente.

¿Qué guarda exactamente, de cada tabla?

A NIVEL DE TABLA

- Cardinalidad: cantidad total de filas.
- Factor de bloqueo: cuántas filas entran en cada bloque de disco.
- Bloques ocupados: cuánto espacio ocupa la tabla.
- Qué índices existen y sobre qué columnas.

A NIVEL DE COLUMNA

- Valores distintos (NDV): cuántos valores únicos tiene la columna.
- Valores mínimo y máximo.
- Selectividad = valores distintos ÷ cardinalidad: qué tan bien filtra esa columna.

Por eso PROYECTO.nombre filtra tan bien: 200 valores distintos en 200 filas es selectividad altísima. En cambio EMPLEADO.cargo, con apenas un puñado de valores distintos en 5.000 filas, filtra mucho menos.

Si estos datos quedan desactualizados, el optimizador puede terminar eligiendo un plan que ya no conviene.

Optimización heurística: reordenar, no reinventar

Árbol de consulta inicial

Sale del análisis sintáctico, sin ninguna optimización. Es correcto, pero ineficiente.

→

se aplican reglas de equivalencia

Árbol de consulta final

Da EXACTAMENTE el mismo resultado, pero con una ejecución mucho más eficiente.

Ambos árboles son expresiones de álgebra relacional. Lo único que cambia entre uno y otro es el orden y la forma de combinar las operaciones — nunca el resultado.

Antes de los símbolos: ¿qué es el álgebra relacional?

¿Qué es?

Un lenguaje formal con un puñado de operadores (filtrar, elegir columnas, cruzar, unir…) para manipular tablas y expresar consultas sobre una base relacional.

¿Para qué sirve?

Es la forma en que el SGBD representa tu consulta por dentro: traduce el SQL a una expresión de álgebra, armada como un árbol (el árbol de consulta que venimos dibujando).

¿Por qué la usamos para optimizar?

Porque las “reglas de oro” que vamos a ver a continuación son transformaciones matemáticamente probadas sobre esas expresiones: garantizan el mismo resultado, con menos costo.

No es matemática abstracta y lejana: el plan que te muestra EXPLAIN es, en el fondo, un árbol de álgebra relacional con nombres físicos (Seq Scan, Hash Join…).

De SQL a álgebra: los operadores básicos

σ

Filtrar filas — σ (sigma)
Es exactamente lo que hace tu cláusula WHERE.

π

Elegir columnas — π (pi)
Es exactamente lo que hace tu cláusula SELECT.

⋈

Cruzar tablas — ⋈ (join)
Es tu FROM con la condición de ensamble entre tablas.

También existen ∪ (unión), ∩ (intersección) y − (diferencia) — las operaciones clásicas de conjuntos, aplicadas a tablas.
No es un lenguaje nuevo: es la forma precisa de escribir la misma idea de "filtrar / elegir columnas / cruzar".

Ahora sí: las 3 reglas de oro para optimizar

Con los operadores ya definidos, podemos escribir con precisión las transformaciones que aplica el optimizador. Elmasri & Navathe y Connolly documentan 12 reglas de equivalencia formales; las resumimos en 3 ideas prácticas que vamos a ver una por una.

1. **Filtrá lo antes posible — σ**
   Aplicá cada condición apenas puedas, sin esperar a cruzar tablas.

2. **Llevá solo lo necesario — π**
   Descartá columnas que no hacen falta lo antes posible.

3. **Acomodá el orden de los cruces — ⋈**
   Los join se pueden reordenar libremente: eso es lo que el optimizador aprovecha.

Cada regla viene con su fórmula formal y un tip práctico para aplicarla.

σ

REGLA DE ORO 1

Filtrá lo antes posible

Si una condición (σ) solo necesita datos de UNA tabla, aplicala apenas leas esa tabla. No esperes a cruzarla con las demás para recién ahí descartar filas.

$$\sigma_c (A \bowtie B) \equiv (\sigma_c (A)) \bowtie B$$

válido cuando c solo usa columnas de A

Tip: cuanto más restrictivo sea el filtro, antes conviene aplicarlo.

π

REGLA DE ORO 2

Llevá solo las columnas que necesitás

Si al final vas a descartar columnas, descartalas lo antes posible. Cada cruce (⋈) que sigue en la cadena va a trabajar con menos datos.

```
πL  (A ⋈ B)
≡
πL  (πL1(A)  ⋈ πL2(B))
```

L1, L2: solo las columnas de A y B que hacen falta

Tip: menos columnas viajando entre operaciones = menos memoria y menos tiempo.

⋈

REGLA DE ORO 3

El orden de los cruces se puede acomodar

Los join (⋈), las uniones (∪) y las intersecciones (∩) no cambian de resultado si cambiás el orden de las tablas, ni la forma en que las agrupás.

$$A \bowtie B \equiv B \bowtie A$$

$$(A \bowtie B) \bowtie C \equiv A \bowtie (B \bowtie C)$$

conmutatividad y asociatividad

Tip: esta libertad es la que le permite al optimizador probar varios órdenes y quedarse con el más barato.

En resumen: frente a una consulta real, el optimizador intenta…

- ✓ Aplicar los filtros (σ) lo antes posible.
- ✓ Empezar por el filtro más restrictivo (el que menos filas deja).
- ✓ Convertir “cruzar todo y filtrar después” en un join directo.
- ✓ Descartar columnas (π) que no hacen falta, apenas puede.

Basado en las reglas de equivalencia de Elmasri & Navathe (Cap. 15) y Connolly et al. (Cap. 18).

⋉

REESCRITURA DE SUBCONSULTAS

IN / EXISTS: el optimizador también las reescribe

Un IN o un EXISTS con subconsulta no se ejecuta fila por fila, como sugiere su sintaxis. El optimizador casi siempre lo reescribe como un cruce: se queda solo con las filas de la tabla externa que tienen al menos una coincidencia en la subconsulta.

```
E.id_empleado IN (...)

        ≡

EMPLEADO ⋉ TRABAJA_EN
```

⋉ = semi-join: filas de A con al menos una coincidencia en B

EXISTS con subconsulta correlacionada se resuelve igual: el optimizador la "decorrelaciona" y la ejecuta, en la práctica, como este mismo semi-join.

El caso traicionero: NOT IN con NULLs

Si la subconsulta puede tener NULL, NOT IN deja de comportarse como esperás.

```
SELECT E.apellido
FROM   EMPLEADO E
WHERE  E.id_empleado NOT IN
       (SELECT T.id_empleado FROM TRABAJA_EN T);
```

✕ NOT IN

Si TRABAJA_EN.id_empleado tiene un solo NULL, cada comparación contra ese valor es UNKNOWN. El resultado: la consulta devuelve cero filas, sin ningún error que avise.

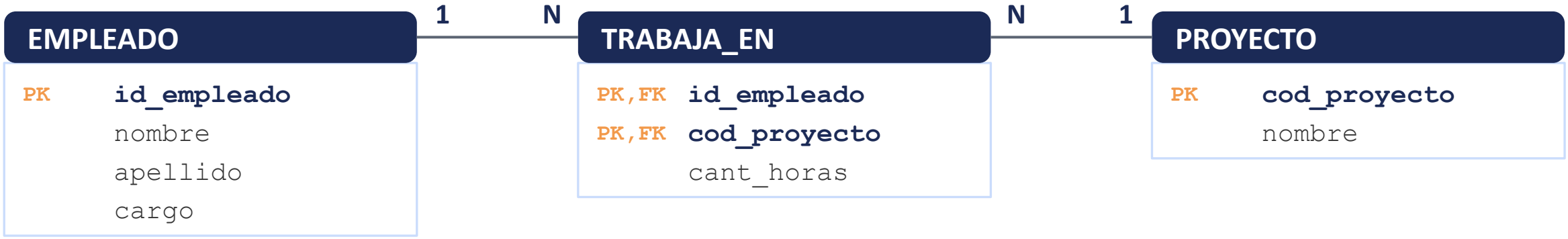
✓ NOT EXISTS

Compara existencia de filas, no valores puntuales — nunca lo afecta un NULL. Es la reescritura que el optimizador siempre puede convertir en un anti-join seguro.

Tip: si no podés garantizar que la columna de la subconsulta sea NOT NULL, preferí NOT EXISTS — es más seguro y, en la práctica, igual de rápido.

Ejercicio — planteemos un caso real

Queremos el apellido de los empleados que trabajan en el proyecto ROBOTICS y tienen cargo de investigador.



PK = clave primaria · FK = clave foránea

```
SELECT E.apellido
FROM   EMPLEADO E, TRABAJA_EN T, PROYECTO P
WHERE  E.id_empleado = T.id_empleado
AND    P.cod_proyecto = T.cod_proyecto
AND    P.nombre = 'ROBOTICS'
AND    E.cargo = 'investigador';
```

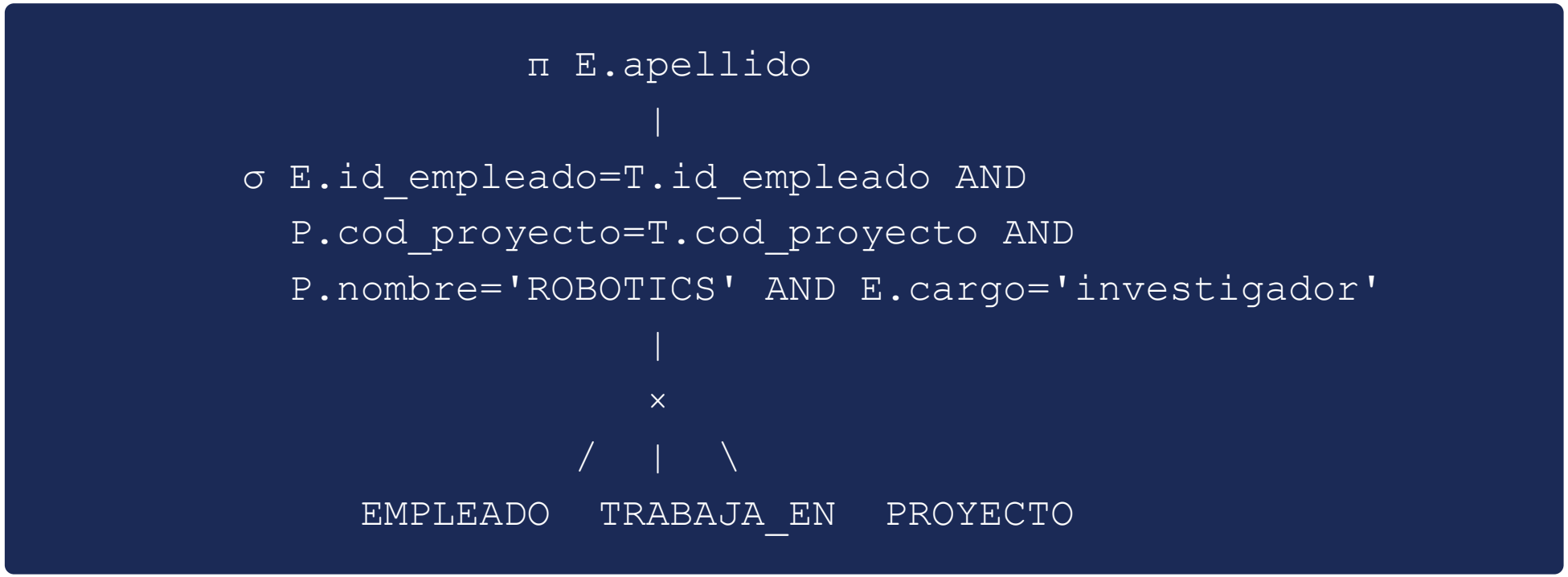
Ejercicio — lo mismo, en álgebra relacional

Es una traducción directa, condición por condición. Todavía no optimizamos nada.

```
π E.apellido (
    σ E.id_empleado=T.id_empleado AND
        P.cod_proyecto=T.cod_proyecto
         AND P.nombre='ROBOTICS' AND E.cargo='investigador' (
      EMPLEADO × TRABAJA_EN × PROYECTO
    )
)
```

Fijate: todavía usamos el producto cartesiano (×), el más costoso de todos los operadores. Ahí es donde entra la optimización.

Ejercicio — el árbol inicial (sin optimizar)



Ineficiente: cruza las 3 tablas COMPLETAS antes de aplicar cualquier filtro.

Ejercicio — aplicamos las 3 reglas de oro

1

Separar el filtro grande en filtros chicos

En vez de un único σ con las 4 condiciones juntas, separamos cada una para que pueda viajar por separado.

2

Empujar cada filtro junto a SU tabla (regla 1)

P.nombre='ROBOTICS' va sobre PROYECTO. E.cargo='investigador' va sobre EMPLEADO. Las condiciones de igualdad entre tablas pasan a ser la condición del join.

3

Elegir qué cruzar primero (regla 3 + heurística)

PROYECTO filtrado por ROBOTICS deja 1 sola fila: conviene cruzarlo primero con TRABAJA_EN, y recién al final incorporar EMPLEADO.

¿Cómo sabe el optimizador que ya encontró un buen plan?

Aplicar las reglas de oro no alcanza por sí solo: el optimizador también estima el costo de cada alternativa y se queda con la mejor. Así se repite el ciclo:

1 Aplica una transformación

Prueba una regla de oro válida sobre el árbol actual y genera un plan candidato.

2 Estima su costo

Usa las estadísticas (cardinalidad, selectividad) para calcular cuánto costaría ejecutarlo.

3 Compara

¿Es más barato que el mejor plan encontrado hasta ahora? Si sí, lo reemplaza.

4 ¿Vale la pena seguir?

Si todavía quedan transformaciones razonables por probar, vuelve al paso 1.

↺ *repite mientras encuentre una alternativa que baje el costo*

Se detiene cuando ya no hay una transformación que mejore el costo, o cuando seguir buscando costaría más de lo que se ahorraría. Ahí se queda con el mejor plan encontrado: el que vamos a comparar con el original a continuación.

Ejercicio — antes y después

ANTES · árbol sin optimizar

```
        π apellido
            |
      σ (4 condiciones)
            |
            ×
          /   |   \
        E     T     P
```

Cruza las 3 tablas completas antes de filtrar.

DESPUÉS · árbol optimizado

```
π apellido
 |
 +- ⋈ (id_empleado)
      |
      +- σ cargo='investigador'
      |    |
      |    +- EMPLEADO
      |
      +- ⋈ (cod_proyecto)
           |
           +- σ nombre='ROBOTICS'
           |    |
           |    +- PROYECTO
           |
           +- TRABAJA_EN
```

¿Y en un motor real? El plan de ejecución

Algunos SGBD permiten inspeccionar la estrategia elegida por el optimizador. Algunos incluso permiten "fijar" el plan.

PostgreSQL te muestra el plan elegido, pero no te deja fijarlo manualmente.

```
EXPLAIN SELECT ...
```

Query execution plan — https://www.postgresql.org/docs/current/sql-explain.html

EXPLAIN: qué te muestra y cómo leerlo, simple

¿Qué te muestra?

- El plan real que eligió el optimizador, no una suposición.
- Qué operación usa en cada tabla: recorrido completo o por índice.
- Qué algoritmo eligió para cada cruce (join).
- Con ANALYZE: el tiempo real de cada paso, no solo el estimado.

¿Cómo leerlo, simple?

- Empezá por la línea más indentada: es lo que se ejecuta primero.
- Cada resultado “sube” y alimenta a la línea de arriba.
- Buscá el nodo con mayor costo o tiempo real: ahí está el cuello de botella.

psql

```
Index Scan using idx_nombre
on proyecto
  (cost=0.28..8.30 rows=1)
  (actual time=0.015..0.016
   rows=1 loops=1)
  Index Cond:
    (nombre = 'ROBOTICS')

Planning Time: 0.082 ms
Execution Time: 0.031 ms
```

Salida real de EXPLAIN ANALYZE para una búsqueda por índice, en un caso cortito.

Leer un plan real: EXPLAIN ANALYZE

Además de mostrar el plan, lo ejecuta de verdad y te dice cuánto tardó cada paso.

```
EXPLAIN ANALYZE SELECT E.apellido
FROM   EMPLEADO E, TRABAJA_EN T, PROYECTO P
WHERE  E.id_empleado = T.id_empleado
AND    P.cod_proyecto = T.cod_proyecto
AND    P.nombre = 'ROBOTICS'
AND    E.cargo = 'investigador';
```

https://www.postgresql.org/docs/15/sql-explain.html

Esto es lo que devuelve, en un motor real

Ejecutamos la consulta de la filmina anterior. Esto es la salida completa de PostgreSQL:

```
Nested Loop  (cost=8.45..16.62 rows=1) (actual rows=1 loops=1)
  ->  Nested Loop  (cost=8.16..15.41 rows=4) (actual rows=4 loops=1)
        ->  Index Scan using idx_proyecto_nombre on proyecto p
              (cost=0.28..8.30)  Index Cond: (nombre = 'ROBOTICS')
        ->  Index Scan using idx_trabaja_en_proyecto on trabaja_en t
              (cost=0.29..7.06)  Index Cond: (cod_proyecto = p.cod_proyecto)
  ->  Index Scan using empleado_pkey on empleado e
        (cost=0.29..0.31)  Index Cond: (id_empleado = t.id_empleado)
        Filter: (cargo = 'investigador'::text)

Planning Time: 0.312 ms      Execution Time: 0.061 ms
```

Los tres accesos son Index Scan — ni un Seq Scan — y arranca por PROYECTO filtrado por 'ROBOTICS', el paso más selectivo. Es, en la práctica, el mismo plan optimizado que armamos a mano: se resuelve en 0,061 ms.

Herramientas para leer un plan sin sufrir

tatiyants.com/pev

Pegás la salida de EXPLAIN y te la convierte en un diagrama interactivo, resaltando los pasos más costosos.

explain.dalibo.com

Lo mismo, con foco en comparar tiempo estimado vs. tiempo real de cada nodo del plan.

¿Qué rol juegan los índices en todo esto?

Cada vez que el optimizador estimó “esto va a filtrar mucho”, en el fondo apostaba a que existe un camino directo hacia esas filas — sin recorrer la tabla entera.

Ese camino directo es, ni más ni menos, un índice. Sin él, cualquier plan sería mucho más caro: cada paso implicaría recorrer la tabla completa.

¿Qué es un índice, exactamente?

Como el índice de un libro

Para encontrar “optimización” en un libro de 400 páginas, no leés página por página: vas al índice del final, que te dice en qué página está, y saltás directo ahí.

En una base de datos

Un índice es una estructura de datos auxiliar que el motor mantiene ordenada por los valores de una o más columnas, para encontrar filas sin recorrer la tabla completa.

¿Para qué sirve?

Para acelerar búsquedas (WHERE), cruces (JOIN) y ordenamientos (ORDER BY) — y para garantizar unicidad, como en las claves primarias.

S **Sin índice → Seq Scan**

El motor recorre la tabla entera, fila por fila, hasta encontrar las que cumplen la condición.

I **Con índice → Index Scan**

El motor salta directo a las filas que cumplen la condición, sin tocar el resto de la tabla.

Crear un índice en PostgreSQL

```
CREATE INDEX idx_nombre ON tabla (col1, col2, ...);

CREATE INDEX idx_nombre ON tabla (col1, col2, ...)
  WHERE predicado;

CREATE INDEX idx_nombre ON tabla
  USING [btree | hash] (col1, col2, ...)
  WHERE predicado;
```

https://www.postgresql.org/docs/current/sql-createindex.html

¿Cómo se materializa un índice, por dentro?

No es magia: es otra estructura

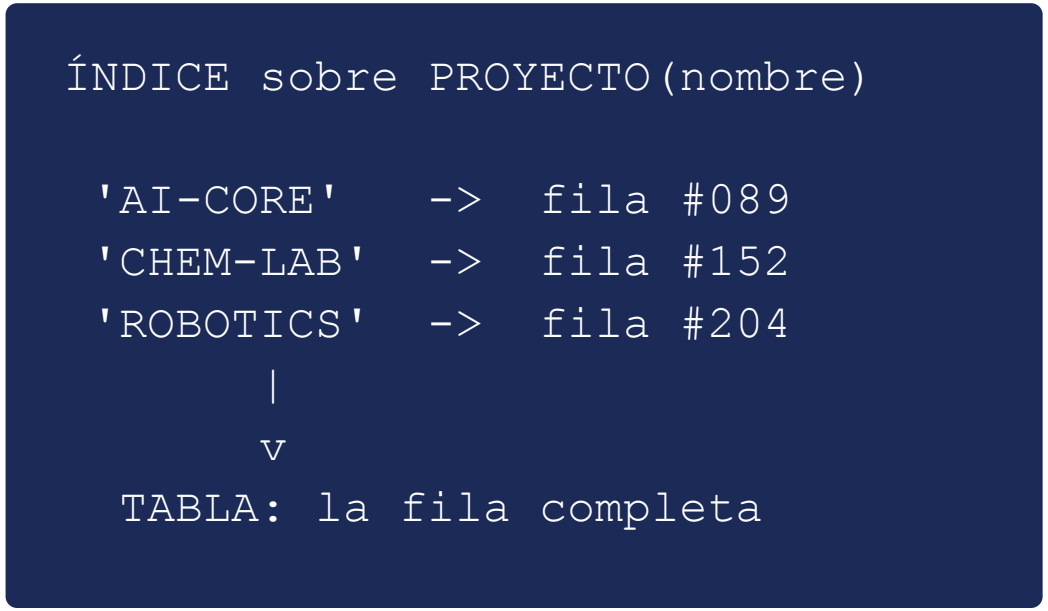
El motor construye y mantiene un objeto aparte, separado de la tabla, que ocupa su propio espacio en disco.

Guarda pares ordenados

Por cada fila, guarda el valor de la columna indexada junto con un puntero a dónde está esa fila completa.

Por eso es más rápido buscar

Buscar en esa estructura chica y ordenada es mucho más barato que recorrer la tabla entera, fila por fila.



B

B-tree

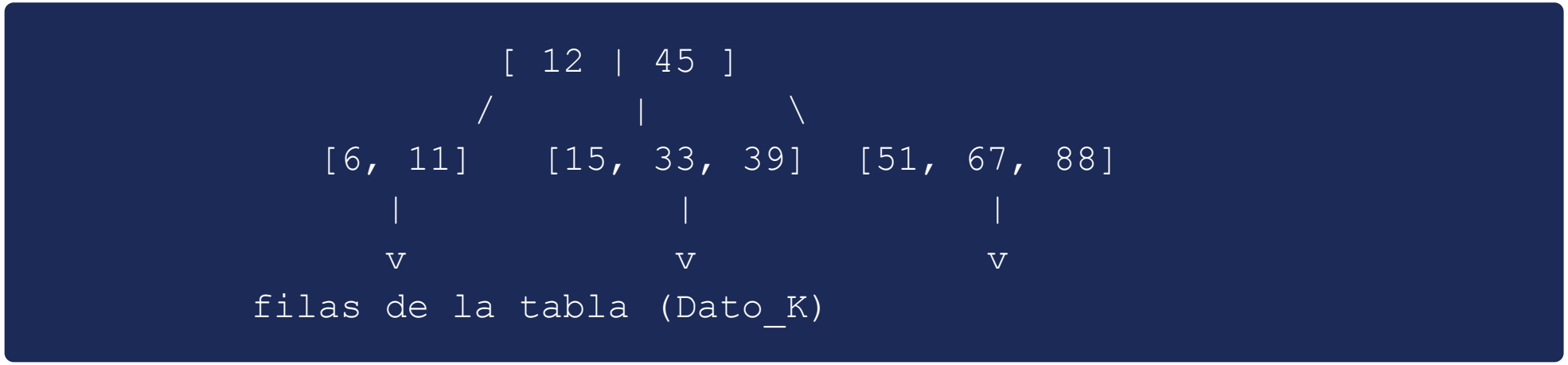
Ordenado: sirve para = y para rangos

H

Hash

Directo: sirve solo para igualdad (=)

Índice B-tree: el más común



- El tamaño del nodo es igual al tamaño de un bloque de disco.
- Por eso entran muchas claves por nodo → el árbol queda “ancho y bajo”.
- Resultado: encontrar cualquier fila toma muy pocas lecturas de disco, incluso con millones de filas.

Índice Hash: directo al bucket

```
'ROBOTICS'  →  función hash  →  bucket 152  →  fila exacta
```

- La función hash calcula directamente en qué “cajón” (bucket) está el dato.
- Es velocísimo para búsquedas por igualdad (=).
- No sirve para rangos: no puede resolver >, <, BETWEEN ni ORDER BY.

Los índices no son gratis

A FAVOR

- Lecturas mucho más rápidas.
- Menos filas escaneadas por consulta.
- Pueden garantizar unicidad (UNIQUE).

EN CONTRA

- Cada INSERT, UPDATE o DELETE debe actualizar también el índice.
- Ocupan espacio en disco.
- Demasiados índices pueden frenar la escritura sin acelerar las lecturas que realmente importan.

Por eso no conviene indexar cada columna “por las dudas”: cada índice es una apuesta a cómo se va a consultar la tabla.

¿Cuándo conviene agregar un índice?

Antes de crear uno, repasá estas cuatro preguntas — juntan varias cosas que ya vimos en la clase.

1

¿Se usa en WHERE, JOIN u ORDER BY?

Si la columna nunca filtra, cruza ni ordena resultados, indexarla no cambia nada.

2

¿Tiene buena selectividad?

Selectividad = valores distintos ÷ cardinalidad. Pocos valores distintos (un booleano, un estado) casi no filtran, aunque tengan índice.

3

¿La tabla es grande?

En tablas chicas, Postgres prefiere Seq Scan aunque exista el índice: ir y volver cuesta más que leer el bloque entero.

4

¿Qué tipo de índice hace falta?

B-tree si necesitás rangos o ORDER BY. Hash solo si siempre vas a filtrar por igualdad exacta.

Después de crearlo, confirmalo con EXPLAIN ANALYZE: si el plan sigue eligiendo Seq Scan, el índice no te está ayudando — revisá selectividad y tamaño de tabla antes de conservarlo.

Para llevarse de esta clase

- El optimizador reescribe tu consulta, no la ejecuta literalmente.
- Filtrar antes y por lo más restrictivo primero es la idea que sostiene casi todo.
- El álgebra relacional es el lenguaje que usa el optimizador por dentro.
- EXPLAIN te deja ver ese plan en un motor real.
- Un índice es lo que hace posible, en la práctica, cada filtro “barato” que vimos.

BIBLIOGRAFÍA

Elmasri, R.; Navathe, S.B. Fundamentals of Database Systems. 7ª Ed. Pearson. (Cap. 17, 18)

Elmasri, R.; Navathe, S.B. Sistemas de bases de datos. Conceptos fundamentales. 2ª Ed. Addison-Wesley Iberoamericana. (Cap. 5 y 16)

Connolly et al. Database Systems: A Practical Approach to Design, Implementation and Management. 2nd Ed. Addison-Wesley (Cap. 18)

Documentación oficial de PostgreSQL — postgresql.org/docs